An Algorithm to Find Anagrams Using Python

George Flanagin
University of Richmond
gflanagin@richmond.edu

Sunday 7th November, 2021

Abstract

Anagrams are fun and difficult. They are an excellent way to explore Python's data structures, and an excuse to develop a few new ones expressly for this purpose. This paper defines an efficient algorithm for finding all anagrams of a given collection of letters using the Python Standard Library for the base code, and the system dictionary, or some other list of words, as a source for allowed words.

This program is the Anagrammar. Its source code and this documentation may be found in https://github.com/georgeflanagin/anagrammatic

Contents

1 Getting started **3**

1.1 Definitions 3

1.2 The big picture of the search 4

2 Notation **4**

3 The Algorithm **7**

3.1 Pruning the dictionary 7

3.2 Trying the words 7

© George Flanagin

3.3 Completeness 8
3.4 Recursion and bookkeeping 9

List of Figures

1 The relationship between anagrams and bit masks 5
2 Notation used to discuss anagrams 6

©George Flanagin

1 Getting started

> *Of course just about every pronounceable combination of five letters has been used to spell or misspell something somewhere, at some point in history.*
>
> Donald E. Knuth, 2011
> *Combinatorial Algorithms, Part 1*, p. 519
> answer to Problem 25 in Section 7

1.1 Definitions

My interest in anagrams started later in life, at 37 years old. An episode of *The Simpsons* named "Lisa's Rival" aired 11 September 1994, and in it Lisa visits a new girl in the school, Allison Taylor. At Allison's house, Lisa is asked to join the Taylors in a specialized game of anagrams in which the names of famous people are rearranged to form descriptive anagrams: *Alec Guinness* becomes *genuine class* in the show's example. Lisa was overwhelmed.

To avoid offending or upsetting readers, I will be using my own name as the basis for the examples in this paper. Mercifully for the contruction of examples, my name consists only of common letters in English, and it has a useful proportion of vowels. Let's call the starting point the *target phrase*, or just the *target*.

To begin, we must have a definition of an anagram. Anagrams always appear in pairs. It makes no sense to say "*George Flanagin* is an anagram," without stating its partner phrase, and the one most suited to Lisa Simpson's game may be *long fearing age*.

The technical properties of an anagram are:

1. Two phrases are anagrams of each other iff all the letters of one phrase are present in the other, the same counts for each letter present. In the case of *George Flanagin*, its anagrams must have exactly three *g*-s, two each of *a*, *e*, and *n*, and one each of *f*, *i*, *l*, *o*, and *r*. Fourteen letters.
2. Each phrase in a pair of anagrams must consist of one or more complete words in some dictionary. Exactly which dictionary is a point of æsthetics, as is whether one should consider single letter words such as `a` in English *and* Spanish, `I` in English, and `y`, `o`, `e`, `u` in Spanish. For the purpose of this paper, we will ignore diacritics.
3. Additional rules are sometimes applied. Common constraints are:
   - ✧ A maximum or minimum number of words in the anagrams of the original phrase, and the related metric, minimum word length.

© George Flanagin

- ✧ The elimination or inclusion of proper nouns in the derived anagrams.
- ✧ A lack of words shared with the original phrase. More discussion of this constraint lies ahead.

Fortunately, none of the above boundaries is difficult to selectively enforce in a program.

1.2 The big picture of the search

At first, the search seems terribly complicated, and it is certainly not trivial. Everyone who has dealt with combinatorics knows how quickly the the number of subsets grows, 16383 in the case of fourteen letters (*i.e.*, $2^{14} - 1$), and the number of partitions, the fourteenth Bell number, is $27,644,437$.[1]

The constraint is the dictionary. The largest one generally used in computing is the list of words that ships as the spelling dictionary with Linux, and it has a mere 479826 words. In any search for anagrams except *the quick brown fox jumped over the lazy dog*, we will eliminate all the words that contain any letter not found in the target. In the case of our example, we can eliminate not only all the words that begin with $[b{-}d, h, j, k, m, n, p, q, s{-}z]$, but all the words that contain one of those letters. Doing no programming at all, we can put the upper limit on the number of words to examine as 2777 if only lower case words are considered, and 5032 if we allow words that normally have capitals in English.[2] These are the upper bounds, because the figures are not derived by looking for the words that have no more of each letter than are found in the target.

For many programmers, the following is a useful abstraction to consider how we go about constructing both *an* anagram, and *all* anagrams. Consider the target phrase and bit-string of the same length. In Figure 1, we see two anagrams, *george+flanagin* and *long+fearing+age*, where each character is (1) or is not (0) used. For both of the anagrams, there is exactly one 1 in each column, and this tabulation is similar to the way that you might search for anagrams with a pile of Scrabble® letters. The equivalent big endian numeric represenations are shown in the right column.

2 Notation

Anagrams suffer from being relatively easy to define in English, yet having no obvious representation in symbols. Given that programming languages are a type of symbolic

[1] The Bell numbers are OEIS sequence A000110, `https://oeis.org/A000110`

[2] These numbers are the results of the two naïve regular expression searches of the Linux spelling dictionary with `^[aegnfilor]\+$` in case sensitive and case insensitive modes.

	g	e	o	r	g	e	f	l	a	n	a	g	i	n	
george	1	1	1	1	1	1	0	0	0	0	0	0	0	0	16128
flanagin	0	0	0	0	0	0	1	1	1	1	1	1	1	1	255
long	1	0	1	0	0	0	0	1	0	1	0	0	0	0	10320
fearing	0	1	0	1	1	0	1	0	1	0	0	0	1	1	5795
age	0	0	0	0	0	1	0	0	0	0	1	1	0	0	268

Figure 1: The relationship between anagrams and bit masks

representation, we will benefit from having a strong symbolic notation for anagrams, their parts, and their construction.

In Figure 2 we can see the basic notation that is invented for the purpose of this discussion. The other symbols used, such as assignment, absolute value, (lexically) less than and non-anagram set operations, are expressed in conventional notation, and will mean what you expect them to.

The notation suggests that our Python representation will need all of these operators/operations:

1. Sorting the letters of a string to make an new string of the same length, which is to say transforming w into $\overrightarrow{w}$.
2. The always useful "partial ordering" operator, $\leq$, so that we can determine if words can be used to make an anagram of the target phrase. From the standpoint of notation, this seems more strict that the subset operator, $\subseteq$.
3. A method to combine two sorted strings into a sorted string result, $w_1 \oplus w_2$, and a method to remove letters from a string, $w_1 \ominus w_2$.
4. A method to filter the useful words in a dictionary to make a new, subset dictionary.
5. A method of bookkeeping to allow us to track the anagrams that have been found, and avoid searching the same path twice.

These operations lead us directly to a discussion of the algorithm to find all anagrams.

© George Flanagin

Symbol	Use and meaning
w, w_n	word or words from a dictionary.
$\mathfrak{S}$, $\mathfrak{T}$	phrases for which are finding anagrams.
$\overrightarrow{w}$, $\overrightarrow{\mathfrak{S}}$	representations of w and $\mathfrak{S}$ where the letters have been sorted. In the programming section, we will be using a standard lexical sort, but this is unimportant as long as the same sort-order is used throughout. If w is `loaf`, then $\overrightarrow{w}$ is `aflo`.
$w \leq \mathfrak{S}$	This expression is true iff w can be constructed from the letters in $\mathfrak{S}$. For example `foal` $\leq$ `georgeflanagin`, and `foal` $\leq$ `loaf`. Note that this expression is true or false without regard to whether the letters in each term have been sorted.
$w_1 \oplus w_2$	The result is a collection of all the letters in the two words, without preserving the order of the letters in each word. The $\oplus$ operator was chosen over + because the bare plus sign is used as a string concatentation operator in many programming languages, including Python.
$w_1 \ominus w_2$	As with the *oplus* operation above, except that we are removing all the letters in w_2 from w_1. This operation is only defined (or meaningful) iff $w_2 \leq w_1$, otherwise in the grammar of anagrams (anagrammar?), the statement is like dividing by zero.
$w_1 \odot w_2$	This expression is true iff w_1 and w_2 are anagrams, so expanding on the above examples, `foal` $\odot$ `loaf` is true, as is: (`long` $\oplus$ `fearing` $\oplus$ `age`) $\odot$ `georgeflanagin`.
r, r_n	r is for remainder, so when we perform a $\ominus$ operation, the result will be a value expressed as an r, so $r_1 = \mathfrak{S} \ominus w_1$
$\mathfrak{D}$, $\mathfrak{D}'$, *etc.*	Throughout, we will use $\mathfrak{D}$ to represent the core dictionary, and $\mathfrak{D}'$ and $\mathfrak{D}''$ to represent derived dictionaries such that $\mathfrak{D}'' \subset \mathfrak{D}' \subset \mathfrak{D}$, in other words, a filter.

Figure 2: Notation used to discuss anagrams

© Georgeflanagin

3 The Algorithm

> *Homer Jay, how do you keep your hair so rich and full?*
> *Lather, rinse, and repeat. Always repeat.*
>
> Homer Simpson
> D'oh-in' in the Wind
> 15 November 1998

Given the small number of qualifying words, and the vast available memory combined with the computational abilities of even bottom-self computers, finding all anagrams of a phrase could be done by brute force. That approach is not very satisfying. Instead, we are searching for elegance and comprehension, two nouns that often appear side-by-side.

3.1 Pruning the dictionary

Our first step is the elimination of all the words that are made from incompatible collections of letters. In our grammar, we seek to construct

$$\mathfrak{D}' := \{w : w \leq \mathfrak{S} \wedge w \in \mathfrak{D}\} \tag{1}$$

This is often a small collection of words, and we know that

$$\forall w : |w| \leq |\mathfrak{S}| \tag{2}$$

As the recursive process proceeds, the string becomes shorter and the pruned dictionary rapidly reduces in size.

3.2 Trying the words

It makes sense to start with the shortest words in $\mathfrak{D}'$, a fact that will guide us when we start the programming in the next section. In fact, we must search the keys of the same length in some fixed order, usually referred to as "sorted order." A conventional lexical sort is OK. Each word in the dictionary will be subtracted from the target phrase, and will leave a remainder that either completes the anagram if it is present in the dictionary, or can be recursively decomposed to find anagrams of the remainder.

© George Flanagin

$$\mathfrak{R} = \{\forall w : \mathfrak{S} \ominus w\} \tag{3}$$

Equation 3 is well suited to the list comprehension construct in Python.

At this point, we should take note of the one-to-many relationship between $\overrightarrow{w}$ and the dictionary words w. Words that are anagrams of each other share the same sorted representation of their letters, so in a key-value look up table, if the keys are of the form $\overrightarrow{w}$, then they must support a n-tuple of one or more w-s as the values.

While programming to use its non-hashable keys requires us to roll our own data structure, it has the advantage that we do not need to try the words in the tuple to make an anagram, we need only concern ourselves with the sorted key.

Consider this specific case: $acer \longrightarrow \{acre, race, care\}$ The sorted representation, $acer$, is meaningless. But we can freely substitute any of the three English words in an anagram that contains one of them. Whether the collection of real words associated with a key has one element or many elements, we only need to bother with the key. Thus, Equation 3 becomes the more manageable expression seen here:

$$\mathfrak{R} = \{\forall \overrightarrow{w} : \mathfrak{S} \ominus \overrightarrow{w}\} \tag{4}$$

We cannot neglect the fact that $\forall w : w \odot w$, or in plain English, every word is an anagram of itself, so we must check to see whether each remainder is a key in the dictionary and a complement of some other key in the same dictionary. So before any recursive decent begins, we must check for the "two word" solution.[3]

3.3 Completeness

How can we be sure that we have found all of the anagrams of $\mathfrak{S}$? Let us consider the collection of keys we will use. It has these properties:

1. The collection consists of sorted strings of characters such that $|\overrightarrow{w_i}| \leq |\overrightarrow{w_{i+1}}|$
2. It is even true that $\overrightarrow{w_i} < \overrightarrow{w_{i+1}}$ because there are no duplicate sorted strings.

[3] In fact, if the original phrase is shorter than the longest words in the dictionary, we must include the "one word" solution. For example, if the target phrase is *gimp count*, there is a one word solution to be found: computing.

© George Hamilton

Let us consider that each collection has a least element, $\overrightarrow{w_0}$. Our first operation is then $r_0 = \mathfrak{S} - \overrightarrow{w_0}$.

What do we know about r_0? We know that $r > \overrightarrow{w_0}$, and that $\forall \overrightarrow{w} : \overrightarrow{w} \in R,$.

3.4 Recursion and bookkeeping

At this point, we have the algorithm reasonably well in mind, if not in hand. We take our collection of remainders and derive $\mathfrak{D}''$ from $\mathfrak{D}'$, and reapply the testing of all the keys in $\mathfrak{D}''$ to $\mathfrak{R}$. The dictionary rapidly becomes small.

Additionally, we can exploit the fact that we are keeping track of the $\overrightarrow{w}$ terms as we go to ensure that we do not test them more than once, and this is where bookkeeping enters the picture. It seems fairly natural to think of this as a forest of n-ary trees, where the root node of each is a $\overrightarrow{w}$ term. For $\overrightarrow{w}$ terms that offer no completion (*i.e.*, dead ends) we can saw these to the ground, and experience has shown that dead ends will constitute the majority of the $\overrightarrow{w}$ terms we try.

©George Flanagin